

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи № 1
з курсу “Теорія обчислень, алгоритми та структури даних”
«Сортування злиттям»

Виконав:
студент групи ФеІ-11
Баран М.Т.
Перевірив:
ас. Баран.М.О.

Львів-2026

Мета роботи

Ознайомитись з методами сортування масивів та реалізувати метод сортування злиттям.

Теоретичні відомості

Привести тут теоретичні відомості то сортуванню масивів злиттям (1-2 сторінки)

Хід роботи

1. Створити бібліотеку *Sort* (файли *Sort.h*, *Sort.cpp*).
2. У бібліотеці *Sort* (файл *Sort.cpp*) реалізуєм функції *Merge* та *MergeSort*

```
void MergeSort(int* A, int first, int last)
{
    if (first < last)
    {
        int mid = (first + last) / 2;
        MergeSort(A, first, mid);
        MergeSort(A, mid + 1, last);
        Merge(A, first, mid, last);
    }
}
```

```

void Merge(int* A, int first, int mid, int last)
{
    int i = first;
    int j = mid + 1;
    int k = 0;
    int* B = new int[last - first + 1];
    while (i ≤ mid && j ≤ last)
    {
        if (A[i] < A[j])
        {
            B[k] = A[i];
            i++;
        }
        else
        {
            B[k] = A[j];
            j++;
        }
        k++;
    }
    while (i ≤ mid)
    {
        B[k] = A[i];
        i++;
        k++;
    }
    while (j ≤ last)
    {
        B[k] = A[j];
        j++;
        k++;
    }
    for (i = first; i ≤ last; i++)
    {
        A[i] = B[i - first];
    }
    delete[] B;
}

```

3. У заголовному файлі *Sort.h* запрограмуємо інтерфейси цих функцій.

```

//Methods Lab_1
void Merge(int* A, int first, int mid, int last);
void MergeSort(int* A, int first, int last);

```

4. Створюємо новий проект *Lab_01* та підключаємо до нього бібліотеку *Sort.h*
5. У функції *main()* проекту реалізуємо можливість введення одномірного масиву даних та відображення результатів сортування

```
int n = 0;

std::cout << "Enter the number of elements: ";
std::cin >> n;

if (n <= 0) {
    std::cout << "Invalid number of elements. Exiting." << std::endl;
    return 1;
}

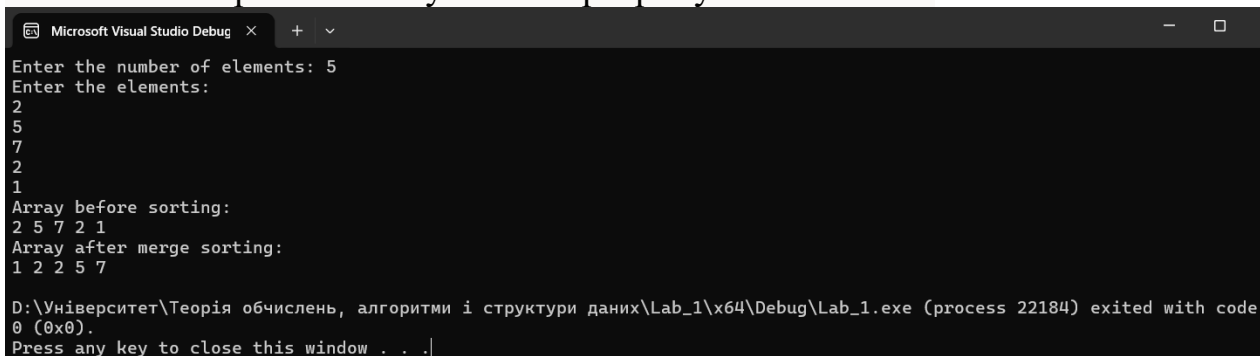
int* A = new int[n];

std::cout << "Enter the elements: " << std::endl;
for (int i = 0; i < n; i++) {
    if (!(std::cin >> A[i])) {
        std::cout << "Помилка: введіть число!" << std::endl;
        std::cin.clear();
        std::cin.ignore(10000, '\n');
        --i;
    }
}

std::cout << "Array before sorting: " << std::endl;
show(A, n);

std::cout << "Array after merge sorting: " << std::endl;
MergeSort(A, 0, n - 1);
show(A, n);
```

6. Компілюємо проект та запускаємо програму на виконання



The screenshot shows the Microsoft Visual Studio Debug console. The program prompts the user to enter the number of elements (5) and then the elements themselves (2, 5, 7, 2, 1). It then displays the array before sorting (2 5 7 2 1) and after merge sorting (1 2 2 5 7). The console also shows the program path and exit code (0).

```
Microsoft Visual Studio Debug  ×  +  v
Enter the number of elements: 5
Enter the elements:
2
5
7
2
1
Array before sorting:
2 5 7 2 1
Array after merge sorting:
1 2 2 5 7
D:\Університет\Теорія обчислень, алгоритми і структури даних\Lab_1\Debug\Lab_1.exe (process 22184) exited with code 0 (0x0).
Press any key to close this window . . .|
```

Висновок

У цій лабораторній роботі я навчився працювати з масивами, застосовувати до них методи сортування, та аналізувати отриманий результат.

Код програми

Sort.h

```
//Methods Lab_1
void Merge(int* A, int first, int mid, int last);
void MergeSort(int* A, int first, int last);
```

Sort.cpp

```
void Merge(int* A, int first, int mid, int last)
{
    int i = first;
    int j = mid + 1;
    int k = 0;
    int* B = new int[last - first + 1];
    while (i <= mid && j <= last)
    {
        if (A[i] < A[j])
        {
            B[k] = A[i];
            i++;
        }
        else
        {
            B[k] = A[j];
            j++;
        }
        k++;
    }
    while (i <= mid)
    {
        B[k] = A[i];
        i++;
        k++;
    }
    while (j <= last)
    {
        B[k] = A[j];
        j++;
        k++;
    }
    for (i = first; i <= last; i++)
    {
        A[i] = B[i - first];
    }
    delete[] B;
}

void MergeSort(int* A, int first, int last)
{
    if (first < last)
    {
        int mid = (first + last) / 2;
        MergeSort(A, first, mid);
        MergeSort(A, mid + 1, last);
        Merge(A, first, mid, last);
    }
}
```

Lab_01.cpp

```
#include <iostream>
#include "Sort.h"
#include <windows.h>
#include <ctime>

int main()
{
    srand(static_cast<unsigned int>(time(0)));
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    int n = 0;

    std::cout << "Enter the number of elements: ";
    std::cin >> n;

    if (n <= 0) {
        std::cout << "Invalid number of elements. Exiting." <<
std::endl;
        return 1;
    }

    int* A = new int[n];

    std::cout << "Enter the elements: " << std::endl;
    for (int i = 0; i < n; i++) {
        if (!(std::cin >> A[i])) {
            std::cout << "Помилка: введіть число!" << std::endl;
            std::cin.clear();
            std::cin.ignore(10000, '\n');
            --i;
        }
    }

    std::cout << "Array before sorting: " << std::endl;
    show(A, n);

    std::cout << "Array after merge sorting: " << std::endl;
    MergeSort(A, 0, n - 1);
    show(A, n);

    delete[] A;
}
```